

《Classifiable RPG》大作业报告

小组名称：饺子&包子

小组成员：焦子冲（2500013224）、包元恺（2400013129）

一、程序功能介绍

我们做的是一个用 Qt 写的桌面小游戏，名字叫《Classifiable RPG》。它表面上是一个回合制 RPG，实际上是想把 C++ 里面向对象的一些知识点做成一关一关的战斗谜题——也就是说，玩家要想打过去，必须先看懂这一关考的是哪个 C++ 面向对象概念，然后用对应的办法去破解它。

程序的界面大致分成三块：左边是战斗场景，画着玩家和敌人的立绘以及各自的血量；右边是一个信息和源码面板，显示当前选中对象的属性和它的“类源码”；下方是命令输入框和战斗日志。整局游戏就是在“读右边的源码 → 想清楚这一关的 C++ 机制 → 在命令框里敲出正确的指令”这样一个循环里推进的，所以它既是一个游戏，也有点像一个能动手操作的 C++ 概念演示器。

游戏的操作方式比较特别：玩家不是点按钮，而是在下方的命令框里输入一行 C++ 风格的指令，形如 `对象.方法(参数)`，比如 `player.attack(bug)`。程序会把这行指令解析出来，然后在战斗里执行。界面右边还有一个源码面板，会实时显示当前选中对象的类定义、成员函数和关键实现，谜题的线索其实就藏在这些源码里——比如看清楚某个怪物的 `onHit` 是怎么被重写的、函数参数到底是传值还是传引用，才能想明白该怎么打。

每一关都有步数上限，敌人也会提前显示它下一回合的意图（要攻击、防御还是强化，以及具体数值），所以玩家可以先看清楚信息再决定怎么走，而不是一味乱试。此外我们还做了血条和护盾的显示、角色立绘、攻击受击治疗和胜利的音效，以及通关后自动进入下一关、卡住时可以看提示等功能。

游戏一共十关，每一关对应一个知识点，从易到难分别是：第一关成员函数调用，第二关访问控制（`public / protected`），第三关析构函数与对象生命周期，第四关对象切片（传值和传引用的区别），第五关静态数据成员和静态成员函数，第六关虚函数与多态，第七关 `dynamic_cast` 向下转型，第八关友元 `friend`，第九关 `shared_ptr` 引用计数和 `weak_ptr`，第十关菱形继承与多重继承的二义性。

为了让这些概念“可玩”，每一关都被包装成一个有明确破解办法的小谜题。举几个例子：第四关有一个“不死之王”，它的 `onHit` 被重写成吸收一切伤害，直接打没用；但只要通过一个传值的接口把它“请下王座”，对象在拷贝时就被切片成普通基类，丢掉了带无敌的

派生部分，于是就能被正常击杀，而换成传引用的接口就不会切片——这一正一反正好把对象切片讲清楚了。第六关把好几个敌人放在一起，它们的 `onHit` 各不相同，有的免疫、有的反弹、有的被打就引爆并波及 Boss，玩家得想清楚先打谁、用谁去触发，才能借多态把 Boss 解决掉。

第七关是一个“拟态魔”，它伪装的时候 `onHit` 会吸收所有伤害，而它真正的弱点是一个只有派生类才有的方法，从基类视角根本看不到、调不到；场上还配了一个“鉴定师”，它的方法内部用 `dynamic_cast` 在运行时识别对象的真实类型，识破之后才能打它的弱点，旁边再放一个“诱饵”用来演示转型失败时会安全地返回空指针。第九关把 `shared_ptr` 的引用计数做成了看得见的血条：一个“命匣”被几个邪教徒共同持有，只要还有人持有它就死不了，玩家必须先把所有持有者清掉、让引用计数归零，命匣才会真正析构。第十关则是一个菱形继承的圣骑士，因为没有用虚继承，它体内其实有两份相互独立的基类子对象，必须用作用域限定分别处理，直接调用会因为二义性而出错。这样原本抽象的语言规则就变成了一个个具体、可操作的战斗目标。

二、项目各模块与类设计细节

整个项目是按照面向对象的思路分层来写的，大致可以分成角色、关卡、指令解析、流程控制和界面显示这几块，各部分之间主要靠 Qt 的信号槽机制来通信。我们尽量让每一层只管自己的事：角色类只负责“一个角色是什么、能做什么”，关卡类负责“这一关有哪些角色、规则是什么”，解析器负责“把玩家敲的话翻译成对某个对象的某次调用”，控制器负责“按回合把这些串起来”，界面类只负责把当前状态画出来。下面分别说一下。

1. 角色类：Creature、Player、Enemy

角色这部分是整个游戏的核心。我们写了一个基类 `Creature`，把所有角色都有的东西放在里面，比如生命值、攻击力、防御、护盾、阵营、类名、意图这些状态，以及 `attack`、`takeDamage`、还有虚函数 `onHit` 等行为；它还提供一个 `classCodeHtml()` 方法，用来把自己的“类源码”拼成文本显示到右边面板上。`Player` 和 `Enemy` 都从 `Creature` 派生，很多关卡里的 Boss 其实就是 `Enemy` 的实例，只是带着不同的类名、方法声明和实现片段而已。

敌人的“意图”我们用一个枚举来表示，分成攻击、防御、强化和未知几种，每种再带一个数值，界面上会显示成一个小徽标，让玩家提前知道敌人下一步要干什么。`Enemy` 还重写了 `classCodeHtml()`，用“基类 + 派生类”的形式把继承关系展示出来，这样玩家在源码

面板上能直接看到某个怪物是从 `Creature` 派生的、又重写了哪些方法。角色的状态一旦变化（掉血、加护盾、改意图），就通过信号通知界面刷新，逻辑和显示是分开的。

多态在这里用得比较多。第六、七、九关都依赖 `onHit` 这个虚函数的动态分发——同样是一次攻击，落在不同对象身上结果可以完全不一样，有的吸收伤害、有的反弹、有的引爆、有的免疫，这正好把虚函数“同一个调用、不同表现”的特点体现出来了。

2. `GameLevel`：关卡和状态管理

`GameLevel` 负责每一关的数据和状态。它给每一关写了一个 `loadLevelN()` 函数，在里面创建这一关需要的角色和道具、设置步数上限和敌人意图，再把它们登记到一个从名字到对象的表里。它还负责记录已经走了几步、用一些标记位和计数器来表示这一关的进度（比如第四关有没有发生切片、第七关有没有识破伪装、第九关的引用计数还剩几），以及判断这一关算赢还是算输。这样设计的好处是，我们不用为每一关都单独写一个新类，用少量通用的状态位加上解析器里的判断，就能表达出每一关各自的机制。

判定输赢的时候也有一些小细节。比如有些关卡里的“道具型”对象（像第四关的王座、第七关的鉴定师和诱饵）并不是真正要击杀的敌人，胜负判断时会把它们排除掉，只看真正的敌人有没有被清完；步数用光还没通关就算失败。每一关还配了一句提示，玩家卡住的时候可以看，提示只点一下思路，不会直接把答案说出来。

3. `CommandParser`：指令解析

玩家输入的那一行字会先交给 `CommandParser`。它用正则把指令分成三种：普通的成员调用 `obj.method(args)`、静态成员函数调用 `Class::method()`（比如第五关的 `Berserker::calm()`），还有带作用域限定的调用 `obj.Base::method()`（比如第十关的 `paladin.Healer::expose(player)`）。解析出来以后再按当前关卡的规则执行，返回这次操作成不成功、要不要消耗步数、日志写什么、播放什么特效。每一关“为什么得这么打”的判断逻辑基本都集中在这里。

举几个具体的例子：第四关里 `throne.seat(king)` 是传值调用，解析器会在这里把不死之王替换成一个被切片过的普通对象，而 `throne.crown(king)` 是传引用，就不会切片；第八关只有“内鬼”这个友元类才能调用金库的私有自毁方法，换成别的角色调用就会被拦下来，并提示它不是 `friend`；第九关里直接攻击命匣是没用的，解析器会告诉玩家得先把持有它的邪教徒清掉；第七关的 `appraiser.reveal(mimic)` 则会触发那次 `dynamic_cast`，成功

识别真身之后才把拟态魔的伪装解除。把这些规则集中放在解析器里，关卡逻辑比较好维护，后面想加新关卡也方便。

4. GameController：回合流程

`GameController` 管的是回合流程。玩家走完一步以后，它会推进回合，调用 `enemyTurn()` 让每个敌人按自己的意图行动，再判断有没有过关、要不要切换到下一关，同时通知界面和音效刷新。敌人行动这块是通用的：控制器遍历场上所有敌人，按它们各自的意图来处理——攻击就对玩家造成对应数值的伤害、防御就给自己加护盾、强化就提升攻击力，个别关卡（比如第十关圣骑士的两个半身，一个治疗一个攻击）会有自己专门的行为。整个回合就是“玩家走一步 → 敌人走一轮 → 刷新显示 → 检查输赢”这样循环下去。

5. 界面与资源

界面这边主要有三个类：`InfoCodePanel` 是右边的信息和源码面板，显示对象的名字、类名、属性和源码，而且源码里的方法签名是可以点的，点一下就把对应指令填进命令框；`BattleSceneView` 配合 `ActorItem` 用 `QGraphicsView` 画出战斗场景，负责背景、立绘的加载缩放、点击和选中的描边、意图徽标等；`CommandPanel` 则是命令输入框和战斗日志。另外还有 `SpriteUtil` 和 `SoundUtil` 两个工具类管贴图和音效，`MainWindow` 把这些面板和控制器拼到一起，`main.cpp` 是程序入口。

源码面板里的内容是各个对象的 `classCodeHtml()` 拼出来的，会把类定义、成员函数签名和关键实现按 HTML 排好版再显示；方法签名做成可点击的，主要是为了降低敲命令的门槛，玩家不用一个字一个字手敲。贴图统一用工具类按相同的高度、圆角和透明边来加载，尽量让不同关卡的立绘看起来风格一致。

三、开发与版本迭代过程

这个项目不是一次写成的，中间改了很多版，这里简单记一下主要的迭代过程。

最开始我们先把基础框架搭起来：定好 `Creature / Player / Enemy` 的继承体系和命令解析的大致流程，把前几关比较简单的关卡（成员函数调用、访问控制、析构）先做出来跑通，确认“读源码—敲指令—看结果”这套玩法走得通，再往上加难度。

接下来主要是打磨关卡。第四关一开始的设计不太直观，玩家不容易看出考点，我们就

把它重做成现在的“对象切片”关，并且专门在源码面板和关卡手册里补充了那个把对象放回场上的引擎函数的说明，让玩家能看明白传值的接口会切片、传引用的接口不会。第五关加了静态成员的“共享狂暴”机制，让一个静态计数在整类敌人之间共享，体现 `static` 成员“全类只有一份”的特点。

美术上我们也花了不少功夫。原版参考的是 AI 厚涂风格的立绘，但我们手上没有现成素材、也没有合适的生成条件，就改成用程序化的方式自己生成贴图，前后调了好几版（我们内部叫 v1 到 v4），一点点修尺寸、配色和整体风格，让各关的立绘尽量统一。工程上也踩过坑，比如有一次构建一直报缓存路径的错，后来发现是 Qt Creator 的本地配置里写死了旧的构建路径、导致 CMake 缓存冲突，清掉那份本地配置就好了，之后我们打包都注意不把个人的本地配置带进去。

后期还做了两件比较大的改动。一是把源码面板的排版重构了一遍：早期每行代码后面都跟一句注释，挤在一起很难读，后来统一改成先把整个类或整个函数的干净代码列出来、再另起一行用注释集中解释，可读性好了很多。二是把原本空着的第七关（之前只放了个训练假人占位）正式做成了 `dynamic_cast` 向下转型关，加了拟态魔、鉴定师和诱饵三个对象，贴图直接复用已有的石巨人、宝珠和金库，没有再新画。最后我们又统一了关卡手册和速查表，把十关各自的最优步数和一些边界情况手动测了几轮，确保都能正常通关。

四、小组成员分工情况

我们小组一共两个人，整体的设计思路、调试和这份报告都是两个人一起商量着完成的，具体的编码任务大致平均分配。包元恺主要负责逻辑这一半，包括 `Creature / Player / Enemy` 这套角色类和战斗核心，以及 `GameLevel` 的关卡构造和 `CommandParser` 的指令解析；焦子冲主要负责流程和显示这一半，包括 `GameController` 的回合控制和敌人行动，以及 `InfoCodePanel`、`BattleSceneView`、`CommandPanel` 这些界面类和贴图音效资源。因为两边都依赖一开始约定好的接口，所以后期联调、十个关卡的设计和测试也是两个人一起做的，工作量基本对半。

具体协作上，我们用 Git 来管理代码，按模块各开各的分支，写完再合并；一开始就把几个关键接口（比如 `Creature` 对外暴露的方法、解析器返回的结果结构）约定清楚，这样两边并行开发不太会冲突。前面提到的美术迭代、工程问题排查和每一关的测试，基本都是两个人一起弄的，所以严格说分工是“逻辑一半、显示一半，公共的部分一起做”，整体工作量比较平均。

五、项目总结与反思

做完这个项目，我们对 C++ 面向对象的内容理解深了不少。继承、多态、虚函数、RAII、智能指针、友元、多重继承这些原来只是课本上的概念，这次为了把它们做成能玩的关卡，必须真正搞清楚它们的行为细节才行，顺便也熟悉了 Qt 的信号槽、QGraphicsView 和资源管理。

特别是把一个知识点“设计成关卡”这件事，逼着我们去想它的边界：比如要做对象切片关，就得先弄明白到底什么时候会切片、切了之后丢的是哪一部分；要做 `dynamic_cast` 关，就得清楚它和 `static_cast` 的区别、失败时为什么会返回空指针。这种“为了讲清楚而先学透”的过程，比单纯做题印象深得多。

过程中也遇到一些麻烦。最主要的是，像对象切片、引用计数归零、`dynamic_cast` 转型失败返回空指针这些，本来是编译期或运行期的现象，怎么在一个回合制游戏里“演”出来让玩家看见，花了我们不少时间，最后是用标记位加上解析器里的分支判断来模拟的，既保证了讲法正确又能交互。另外右边源码面板的排版也改过几次，一开始每行代码后面都跟一句注释，看着很乱，后来改成先把整段代码完整列出来、再另起一行用注释统一解释，看起来就清楚多了。

不足的地方也有：有些贴图是我们自己临时生成的，风格不太统一；游戏也还没有做存档、撤销这些功能，测试主要靠手动试，没有写自动化的测试。如果以后还有时间，可以再补几关，比如模板、运算符重载、异常处理这些知识点其实都能设计成新关卡。总的来说，这次大作业让我们把一学期学的 C++ 比较完整地用了一遍，收获挺大。